HRSG 계산기·체크리스트 키트 — 팀 2

팀 2 페이지의 계산기 3개 탭 과 플랜트 데이터 체크리스트 에 붙일 로직과 임시 데이터입니다.

두 팀이 같은 코어를 씁니다. `calc.js`, `checklist.js`, `data/`, 테스트, FORMULAS·CHECKLIST 문서는 팀 1 키트와 똑같습니다. 팀마다 다른 것은 이 README 의 연결표 뿐입니다. 모든 수치는 가상(Mock) 값입니다.

구성

파일	내용
`calc.js`, `checklist.js`	계산기·체크리스트 함수 — JavaScript 판 (외부 라이브러리 없음)
`calc.py`, `checklist.py`	같은 함수 — Python 판 (표준 라이브러리만)
`data/calculator.json`	GT 등급 3개, 예시 운전 상태 1개, 경제성, 세정 공법 4개, 핀치 설계값, 계산 상수
`data/checklist.json`	카테고리 5개, 항목 49개 (필수 16)
`FORMULAS.md`, `CHECKLIST.md`	계산식 / 체크리스트 항목·상태·함수
`calc.test.mjs`, `checklist.test.mjs` / `test_calc.py`, `test_checklist.py`	기대값 테스트 — JavaScript 15개 (Node 18 이상) / Python 16개 (Python 3.8 이상, 추가 설치 없음)
`example.html`	어댑터 → 검사 → 계산 → 카드 의 가장 작은 예시 (JavaScript)
`README.pdf`, `CLAUDE.md`	이 문서의 PDF 판 / Claude Code 작업 규칙

팀 2 화면 연결표

함수 이름 (JavaScript ↔ Python)

두 판은 같은 식·같은 결과입니다(무작위 입력 200개로 비교해 차이 0). 결과 모양도 같아서, 이 문서의 `r.kpi.power_loss_mw` 는 Python 에서 `r['kpi']['power_loss_mw']` 입니다.

JavaScript (`calc.js`, `checklist.js`)	Python (`calc.py`, `checklist.py`)
`fetch('data/calculator.json')` 등으로 읽기	`load_calculator_data()`, `load_checklist_data()`
`defaultInputs` · `designInputs` · `validateInputs` · `calculate`	`default_inputs` · `design_inputs` · `validate_inputs` · `calculate`
`compareMethods(inp, D, 'payback')` · `lossCurve(inp, D, { param })` · `efficiencyTrend`	`compare_methods(inp, D, 'payback')` · `loss_curve(inp, D, param=..)` · `efficiency_trend`
`keyIndicators` · `pinchSection` · `formatKrw`	`key_indicators` · `pinch_section` · `format_krw`
`validateProgress` · `kpis` · `byCategory` · `summary` · `distribution` · `matrix`	`validate_progress` · `kpis` · `by_category` · `summary` · `distribution` · `matrix`
`filterItems(C, prog, { requiredOnly })` · `priorityLabel` · `detailLine` · `measureCell`	`filter_items(C, prog, required_only=True)` · `priority_label` · `detail_line` · `measure_cell`
`emailDraft(C, prog, { ids })` · `exportRows` · `importReply` · `resetProgress`	`email_draft(C, prog, ids=..)` · `export_rows` · `import_reply` · `reset_progress`

계산기 입력 (내 화면 → 키트 입력 inp)

팀 2 화면	키트 입력	비고
가스터빈 모델 (F/H/E-Class GT (Mock))	`equipment.gt_class`	`"F"` / `"H"` / `"E"`. 정격·설계값은 등급에서
HRSG 차압 ΔP (mbar)	`equipment.hrsg_dp_mbar`	단위는 키 이름으로
GT 배압 (mbar)	`equipment.gt_bp_mbar`	
스택 온도 (°C)	`equipment.stack_c`	
GT 부하율 · 배기온도 · 배기가스 유량 · 대기온도	`equipment.load_pct` · `gt_exhaust_c` · `exhaust_flow_kgs` · `ambient_c`	
고급 입력: 섹션별 드럼 압력 · 증발기 출구 가스 · 절탄기 출구 급수	`sections.HP` / `IP` / `LP` 의 `drum_barg` · `evap_outlet_gas_c` · `econ_outlet_water_c`	데이터 없는 섹션은 `null`
HP 스팀 생산량 (t/h)	`hp_steam_tph`	
전력 단가 (원/MWh) · 연간 운전 시간 (h)	`economics.price_krw_per_mwh` · `economics.annual_hours`	월간은 없으면 ÷12
세정 공법 4개	`methods` (데이터 파일 그대로)	

계산기 결과 (키트 결과 → 내 화면)

팀 2 화면	키트 결과 (`r = calculate(inp, D)`)
탭 1 지표 3개 판정 · 종합 상태 · 게이지	`r.indicators` (값·설계·주의·위험·min·max·level) · `r.overall_level`
총 출력 손실 (GT·ST) · 연간 손실	`r.loss` · `r.annual_loss_krw`
세정 시 연간 회수 · 손실 대비 회수율 (기계식 세정)	`r.default_method.annual_recovery_krw` · `r.default_method.recovery_vs_loss`
열회수 효율 (현재·세정 후·설계) · 오염 지수 · HRSG 도식	`r.efficiency` · `Math.round(r.index.value)` · `r.display_fouling`
열회수 효율 추이 · 차압 변화에 따른 손실 감도	`efficiencyTrend(inp, D)` · `lossCurve(inp, D, { param: 'hrsg_dp' })`
탭 2 공법별 비용 · 정지일수 · 회수율 · 회수기간 · 순편익	`r.methods.rows` 의 `total_investment_krw` · `outage_days` · `recovery_ratio` · `payback_months` · `annual_net_krw`
「순편익 최대」	`r.methods.max_net_benefit_id` (null 이면 표시 안 함)
탭 3 핀치·어프로치 섹션 HP/IP/LP · KPI 10개	`keyIndicators(inp, D)` 의 `sections` · `kpi`
초기화 (설계값)	`designInputs(D, 'F')` — 손실 0 / 예시 운전 상태는 `defaultInputs(D)`

체크리스트

팀 2 화면	키트
「전체 진행률 0% (0/49)」 「필수 0/16」 · 분류별 막대·탭	`summary()` (`all` · `required` · `byCategory`)
왼쪽 동그라미 체크	체크 = `supply: 'received'`
상태 드롭다운 대기 / 요청함 / 수령 / 해당 없음	`supply`: `not_entered` / `requested` / `received` / `na`
유형 배지 · 우선순위 배지 (필수/권장/선택)	`C.kinds[item.kind]` · `priorityLabel(C, item, 'simple')`
「계산기 입력에 사용됨」	`item.calc_input` 이 있으면 표시
검색 · 「필수」 · 「미완료」 · 「초기화」	`filterItems({ query, category, requiredOnly, incompleteOnly })` · `resetProgress()`
설명 줄	`detailLine(item)`
「이메일 텍스트 복사」	`emailDraft(C, prog)` (미수령 항목 전체)
탭 3 「체크리스트에서 데이터 요청」	`pinchSection().checklist_category = 'B'`

알아 둘 점

- 체크리스트는 5개 분류(A 가스터빈/운전 · B HRSG 열역학/설계 · C 세정/유지보수 · D 경제성 · E 확장), 49개 항목이며 ID 는 `A-01` 형식입니다.
- 팀 1 이 쓰는 「검증 필요」·검증 상태·검토 체크 필드도 있지만 쓰지 않아도 됩니다.
- 세정 공법 비용에 정지 손실(정지일수 × 정격 × 전력 단가 × 2%)이 들어갑니다. 회수기간은 월간 편익 기준입니다.
- `defaultInputs(D)` 는 예시 운전 상태(손실 있음), 「초기화」에 쓸 설계 상태(손실 0)는 `designInputs(D, 'F')` 입니다.

내 프로젝트에 넣기 (Claude Code, 한 번에)

1. 키트 넣기: 받은 `hrsg_calc_kit_team2.zip` 을 내 웹 프로젝트 폴더 안에 복사합니다. 이 PDF 를 보려고 이미 압축을 풀었다면 `hrsg_calc_kit_team2` 폴더를 통째로 넣어도 됩니다.
2. 저장해 두기: git 을 쓰고 있다면 지금 상태를 commit 해 둡니다. 결과가 마음에 들지 않으면 되돌릴 수 있습니다.
3. Claude Code 열기: 그 폴더에서 Claude Code 를 엽니다(VS Code 로 폴더를 열고 Claude Code 실행, 또는 터미널에서 그 폴더로 이동해 `claude`).
4. 프롬프트 붙여 넣기: 아래 프롬프트 하나를 그대로 붙여 넣습니다. 파일 수정 허가를 물으면 허용합니다. Claude Code 가 아래 「작업 목록」을 모두 하고, 「기대값」으로 스스로 확인한 뒤 결과 표를 보여 줍니다.

프롬프트 1 · 처음 한 번 붙여 넣기 — 상자 안의 글 전체를 복사해 붙여 넣기

```
이 프로젝트에 hrsg_calc_kit_team2.zip 이 있으면 hrsg_calc_kit_team2 폴더로 풀고(이미 폴더가 있으면 그대로 써 줘),
폴더 안의 파일 구조는 바꾸지 마.
그 폴더의 README.md, FORMULAS.md, CHECKLIST.md, CLAUDE.md 를 읽고,
먼저 이 프로젝트의 언어와 프레임워크를 확인해서 알려 줘. JavaScript/TypeScript 면 calc.js · checklist.js 를,
Python(Streamlit·Flask 등)이면 calc.py · checklist.py 를 그대로 쓰고(README 「함수 이름」 표),
그 밖의 언어면 calc.js · checklist.js 를 그대로 옮긴 뒤 키트 테스트와 같은 입력에서 같은 기대값이 나오는지 확인해 줘.
그 언어의 키트 테스트가 통과하는지 확인해 줘(JavaScript: node --test hrsg_calc_kit_team2/calc.test.mjs
  hrsg_calc_kit_team2/checklist.test.mjs,
Python: python -m unittest discover -s hrsg_calc_kit_team2 -p "test_*.py").
그다음 README 의 「팀 2 화면 연결표」와 「작업 목록」대로 내 계산기 3개 탭과 체크리스트 화면에 키트의 데이터와
  로직을 모두 연결해 줘.
- 내 화면 상태를 키트 입력으로 바꾸는 어댑터 함수 하나를 만들고, validateInputs() 의 오류·경고가 0개가 되게 해 줘.
- 화면 구성과 디자인은 바꾸지 말고, 내 화면에 없는 요소는 새로 만들지 말고 건너뛰어 줘.
- hrsg_calc_kit_team2 폴더 안의 파일은 고치지 마.
끝나면 README 의 「기대값」 표의 값이 화면에 나오는지 하나씩 확인해서 (항목, 기대값, 내 화면 값, 일치 여부) 표로
  보여 주고,
연결하지 못했거나 건너뛴 것이 있으면 이유와 함께 알려 줘.
```

값이 다르거나 오류가 나면 이어서 붙여 넣습니다.

프롬프트 2 · 값이 다르거나 오류가 날 때만 — 상자 안의 글 전체를 복사해 붙여 넣기

```
화면 값이 README 의 기대값과 다르거나 오류가 나. 어댑터가 만든 inp 를 콘솔에 출력하고 validateInputs(inp, D) 의
errors 와 warnings 를 보여 준 다음, README 연결표와 비교해서 원인을 설명해 줘. 코어 파일은 고치지 말고 어댑터만
  고쳐 줘.
```

작업 목록

계산기

1. 키트 불러오기: `calc.js` 또는 `calc.py` 와 `data/calculator.json` (Python 은 `load_calculator_data()`). 가스터빈 모델 목록 = `D.gt_classes`. 처음 화면 = `defaultInputs(D)`, 「초기화」 = `designInputs(D, classId)` (선택한 등급). 입력 변경 때 `calculate(inp, D)`.
2. 탭 1: 지표 3개 판정·게이지 = `r.indicators`, 종합 상태 = `r.overall_level`, 총 출력 손실 = `r.loss` (GT·ST·합계), 연간 손실 = `r.annual_loss_krw`, 세정 시 연간 회수·손실 대비 회수율 = `r.default_method`, 열회수 효율 = `r.efficiency`, 오염 지수 = `Math.round(r.index.value)`. 금액은 `formatKrw`.
3. 탭 1 그래프: 열회수 효율 추이 = `efficiencyTrend(inp, D)` (13개 점 + 세정 후), 차압 변화에 따른 손실 감도 = `lossCurve(inp, D, { param: 'hrsg_dp' })` (`value.mbar`, `loss_mw`, 현재·주의·위험), HRSG 도식 색 = `r.display_fouling` (0~1).

4. 탭 2: 표 = `compareMethods(inp, D).rows` — 비용 `total_investment_krw` · 정지일수 · 회수율 · 회수기간(null → 「산출 불가」) · 순편익 `annual_net_krw`, 열 머리글로 정렬. `max_net_benefit_id` 공법에 「순편익 최대」 배지(null 이면 없음). 「권장」이라고 쓰지 않음.
5. 탭 3: 고급 입력 = `inp.sections` (HP/IP/LP), `inp.hp_steam_tph`. 결과 = `keyIndicators(inp, D)` — 섹션 카드 `sections`, KPI 카드 10개 `kpi`. 「데이터 필요」 카드의 「체크리스트에서 데이터 요청」 → 체크리스트를 `checklist_category('B')` 분류로 열기.

체크리스트 (플랜트 데이터 체크리스트)

6. 항목 = `data/checklist.json` 의 49개. 진행 상태는 `{ "A-01": { supply, note } }` 로 저장(브라우저 앱은 localStorage, Python 앱은 세션 상태와 JSON 파일 — 예: Streamlit `st.session_state` + `progress.json`)하고, 불러올 때 `validateProgress()` 오류가 있는 항목만 처음 상태로.
7. 진행률 카드 = `summary()` 의 `all` · `required`, 분류별 막대·탭 = `byCategory()`.
8. 항목 행: 설명 줄 `detailLine()`, 유형 배지 `C.kinds[item.kind]`, 우선순위 배지 `priorityLabel(C, item, 'simple')`, `calc_input` 이 있으면 「계산기 입력에 사용됨」. 동그라미 체크 = `received`, 상태 드롭다운 대기 `not_entered` · 요청함 `requested` · 수령 `received` · 해당 없음 `na` (키로 저장). 검색·「필수」·「미완료」 = `filterItems()`, 「초기화」 = `resetProgress()`.
9. 「이메일 텍스트 복사」 = `emailDraft(C, progress)` (미수령 항목 전체) — 복사만, 메일은 보내지 않음.

기대값

화면	상태	기대값
탭 1	처음 화면 (예시 운전 상태)	GT 배압 30 정상 · HRSG 차압 30 주의 · 스택 95 정상 · 종합 주의 · 총 출력 손실 1.89 MW (GT 1.08 · ST 0.81) · 연간 손실 22.68억 원 · 기계식 세정 회수 0.85 MW, 연간 10.21억 원, 회수율 45% · 열회수 효율 86.3% → 세정 후 86.7% (설계 87.2%) · 오염 지수 41
탭 1	초기화 (F-Class 설계)	손실 0.00 MW · 오염 지수 0 · 정상
탭 1 그래프	처음 화면	효율 추이 87.2% → 86.3% (13개 점), 세정 후 86.7% · 감도 곡선 15 mbar 0.81 MW → 40 mbar 4.05 MW, 주의 27.5 · 위험 32.5, 현재 30
탭 2	처음 화면	화학 세정 「순편익 최대」 11.86억 원 · 비용 기계식 3.25억 / 고압수 4.04억 / 드라이아이스 4.46억 / 화학 7.42억 원 · 회수기간 3.54 / 3.30 / 3.12 / 4.27개월
탭 2	초기화	순편익 모두 마이너스, 회수기간 「산출 불가」, 배지 없음
탭 3	처음 화면	HP 핀치 8.0 °C · 어프로치 5.0 °C 정상, IP·LP 데이터 필요 · 종합 「설계 범위를 벗어난 지표가 있습니다」 · 스팀 270 t/h · 열회수 효율 86.3% (-0.85%p) · 손실 1.89 MW · 세정 후 회복 0.85 MW (45%) · 연간 손실 22.68억 원 · 세정 비용 3.25억 원 (기계식 세정) · 투자 회수 기간 3.54개월
체크리스트	처음	「전체 진행률 0% (0/49)」 · 「필수 0/16」 · A 0/6 · B 0/13 · C 0/9 · D 0/3 · E 0/18 · 「필수」 16개 · 「핀치」 검색 5건
체크리스트	A-01 체크 / B-13 해당 없음	전체 2% (1/49) / 분모 48 · 새로고침해도 유지
이메일 텍스트	처음	「HRSG 성능 진단을 위해 아래 데이터를 요청드립니다.」 로 시작, 〔A. 가스터빈/운전〕 부터 분류별 · 수령한 항목은 빠짐

칼럼이 달라서 생기는 오류 막기

- `validateInputs(inp, D)`: 빠진 값·숫자가 아닌 값·모르는 GT 등급·같은 압력을 두 단위로 넣은 경우는 오류, 모르는 이름·단위가 의심되는 값은 경고.
- `calculate()` 는 오류가 있으면 계산하지 않고, 무엇이 빠졌는지 적힌 오류를 냅니다(화면에 NaN 이 나오지 않음). `<input>` 의 문자열 값은 숫자로 바꿔 계산합니다.
- `validateProgress(C, progress)`: 모르는 항목 ID(예: `gt_model`), 화면 글자를 그대로 저장한 상태 값을 알려 줍니다.

규칙

- 코어(`calc.js`·`calc.py`, `checklist.js`·`checklist.py`, `data/`)는 고치지 않습니다. 이름·단위 차이는 어댑터에서 바꿉니다.
- 공법 비교에서 가장 좋은 공법은 「권장」이 아니라 「순편익 최대」로 표시합니다.
- 발전소 이름·측정값·메모는 LLM 에 보내지 않습니다. 이 키트를 외부에 다시 배포하지 않습니다.
- `example.html`(JavaScript 예시)은 키트 폴더에서 `npx serve .` 또는 `python3 -m http.server` 로 엽니다.